

Reinforcement Learning

TD Learning Control

Min-hwan Oh

Seoul National University

Temporal-Difference (TD) Control

- Last lecture, we learned TD prediction.

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

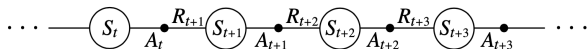
- This lecture, we will study algorithms for TD control.
- These methods follow the pattern of generalized policy iteration (GPI) using TD prediction.

Sarsa: On-policy TD Control

- Learn an action-value function $q_{\pi}(s, a)$ rather than a state-value function $v_{\pi}(s)$.
 - ▶ Recall Q is more useful than V because?

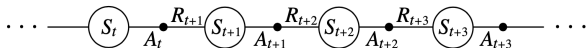
Sarsa: On-policy TD Control

- Learn an action-value function $q_\pi(s, a)$ rather than a state-value function $v_\pi(s)$.
 - ▶ Recall Q is more useful than V because? we do not have a model!



Sarsa: On-policy TD Control

- Learn an action-value function $q_\pi(s, a)$ rather than a state-value function $v_\pi(s)$.
 - ▶ Recall Q is more useful than V because? we do not have a model!



- The convergence of state values v_π under TD(0) also apply to the “Sarsa” update for action values

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

- Update is done after every transition from a nonterminal state.

Sarsa Pseudocode

- Estimate q_π for the behavior policy π , and at the same time change π toward greediness with respect to q_π

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Choose A from S using policy derived from Q (e.g., ε -greedy)

 Loop for each step of episode:

 Take action A , observe R, S'

 Choose A' from S' using policy derived from Q (e.g., ε -greedy)

$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma Q(S', A') - Q(S, A)]$

$S \leftarrow S'; A \leftarrow A';$

 until S is terminal

Greedy in the Limit with Infinite Exploration (GLIE)

- All state-action pairs are explored infinitely many times

$$\lim_{t \rightarrow \infty} N_t(s, a) = \infty$$

- The policy converges on a greedy policy

$$\lim_{t \rightarrow \infty} \pi_t(a|s) = \mathbb{1} \left\{ a = \arg \max_{a'} Q_k(s, a') \right\}$$

Greedy in the Limit with Infinite Exploration (GLIE)

- All state-action pairs are explored infinitely many times

$$\lim_{t \rightarrow \infty} N_t(s, a) = \infty$$

- The policy converges on a greedy policy

$$\lim_{t \rightarrow \infty} \pi_t(a|s) = \mathbb{1} \left\{ a = \arg \max_{a'} Q_k(s, a') \right\}$$

- For example, ϵ -greedy is GLIE if ϵ decreases to zero, e.g., $\epsilon_t = \frac{1}{t}$.

Convergence of Sarsa

Theorem

Sarsa converges to the optimal action-value function, $Q(s, a) \rightarrow q_(s, a)$, under the following conditions:*

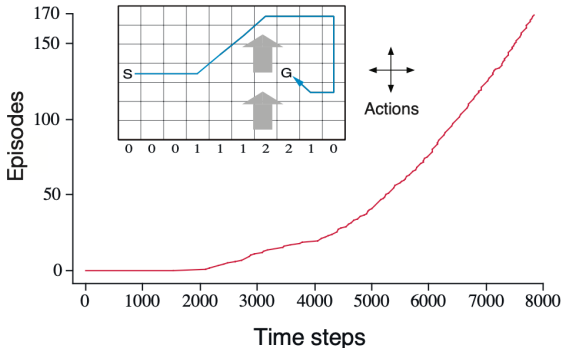
- $\pi_t(a|s)$ satisfies GLIE
- step-size α_t satisfies stochastic convergence equations:

$$\sum_{t=1}^{\infty} \alpha_t = \infty$$

$$\sum_{t=1}^{\infty} \alpha_t^2 < \infty$$

Sarsa in Windy Gridworld

- Undiscounted episodic task, with constant rewards of -1 until the goal state is reached
- In the middle region the resultant next states are shifted upward by a “wind,” the strength of which varies from column to column



Q-learning: Off-policy TD Control

- Q-learning (Watkins, 1989) update rule:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

- ▶ compare with Sarsa update:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

Q-learning: Off-policy TD Control

- Q-learning (Watkins, 1989) update rule:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

- ▶ compare with Sarsa update:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

- Approximates q_π , the optimal action-value function, independent of the policy being followed
- **Requirement:** all pairs must continue to be updated
 - ▶ e.g., use ϵ -soft policy

Q-learning Pseudocode

- The policy still has an effect in that it determines which state–action pairs are visited and updated.
- But as long as all pairs continue to be updated, correct convergence is guaranteed.

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

 Choose A from S using policy derived from Q (e.g., ε -greedy)

 Take action A , observe R, S'

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

$S \leftarrow S'$

 until S is terminal

Why is Q-learning Off-policy?

- **Off-policy**: evaluate or improve a target policy from data generated by a different behavior policy

Why is Q-learning Off-policy?

- **Off-policy**: evaluate or improve a target policy from data generated by a different behavior policy
- The target policy π is **greedy** w.r.t. $Q(S, A)$:

$$\pi(a|S_t) = \arg \max_{a'} Q(S_t, a')$$

- The behaviour policy b is not the same as π , e.g., ϵ -greedy.

Why is Q-learning Off-policy?

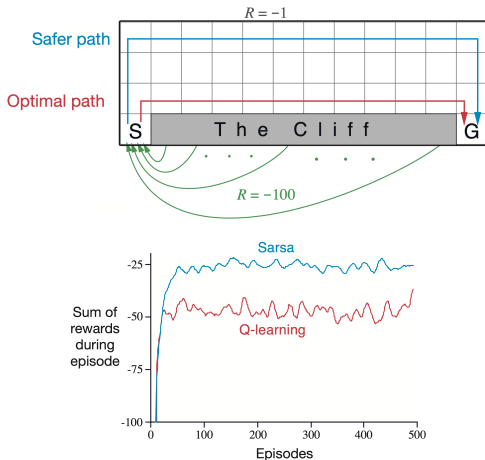
- **Off-policy**: evaluate or improve a target policy from data generated by a different behavior policy
- The target policy π is **greedy** w.r.t. $Q(S, A)$:

$$\pi(a|S_t) = \arg \max_{a'} Q(S_t, a')$$

- The behaviour policy b is not the same as π , e.g., ϵ -greedy.
- Q-learning target for $A' \sim \pi(\cdot|S_t)$

$$\begin{aligned} R_{t+1} + \gamma Q(S_{t+1}, A') &= R_{t+1} + \gamma Q(S_{t+1}, \arg \max_{a'} Q(S_{t+1}, a')) \\ &= R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') \end{aligned}$$

Sarsa vs. Q-learning: Cliff Walking Example



- Although Q-learning actually learns the values of the optimal policy, its online performance is worse than that of Sarsa.
- What if ϵ were gradually reduced?

Sarsa vs. Q-learning: Convergence Guarantees

- Sarsa depends on policy's dependence on action value function Q
 - ▶ Sarsa's policy must converge to greedy (GLIE)
 - ▶ Step-size need to satisfy stochastic convergence equations
- Q-learning's value function converges to true value function q_*
 - ▶ as long as all state-action pairs visited infinitely
 - ▶ and step-size satisfies stochastic convergence equations

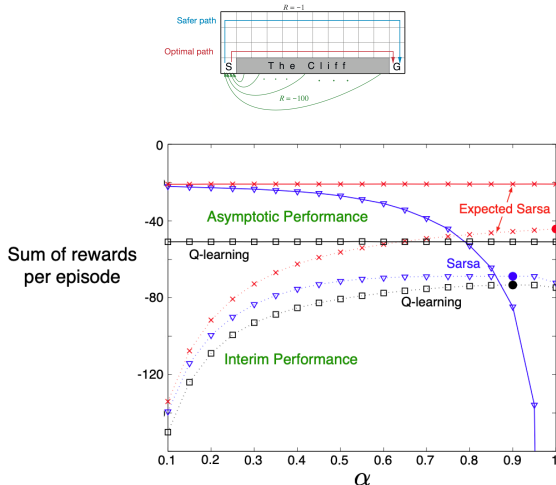
Expected Sarsa

- Take into account how likely each action is under the current policy
- Expected Sarsa update rule:

$$\begin{aligned} Q(S_t, A_t) &\leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \mathbb{E}_{\pi}[Q(S_{t+1}, a) \mid S_{t+1}] - Q(S_t, A_t)] \\ &\leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \sum_a \pi(a|S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t)] \end{aligned}$$

- Compare with Sarsa and Q-learning update:
 - ▶ Sarsa: $Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$
 - ▶ Q-learning $Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$
- Expected Sarsa is more computationally complex than SARSA, but eliminates variance due to random selection of A_{t+1} .

Expected Sarsa vs. Sarsa vs Q-learning in Cliff-Walking



- Asymptotic performance: an average over 100,000 episodes
- Interim performance: an average over the first 100 episodes

Is Expected Sarsa On-policy or Off-policy?

- In the cliff-walking example, Expected Sarsa was used on-policy.

Is Expected Sarsa On-policy or Off-policy?

- In the cliff-walking example, Expected Sarsa was used on-policy.
- But, in general, can use a policy different from the target policy π to generate behavior.

Is Expected Sarsa On-policy or Off-policy?

- In the cliff-walking example, Expected Sarsa was used on-policy.
- But, in general, can use a policy different from the target policy π to generate behavior.
- Hence, expected Sarsa is an off-policy algorithm.
- e.g., Q-learning is a special case of expected Sarsa!

Is Expected Sarsa On-policy or Off-policy?

- In the cliff-walking example, Expected Sarsa was used on-policy.

Is Expected Sarsa On-policy or Off-policy?

- In the cliff-walking example, Expected Sarsa was used on-policy.
- But, in general, can use a policy different from the target policy π to generate behavior.

Is Expected Sarsa On-policy or Off-policy?

- In the cliff-walking example, Expected Sarsa was used on-policy.
- But, in general, can use a policy different from the target policy π to generate behavior.
- Hence, expected Sarsa is an off-policy algorithm.
- e.g., Q-learning is a special case of expected Sarsa!

Relationship Between TD and DP

TD (Sample backup)	DP (MDP backup)
TD learning: $R + \gamma V(S')$	Policy evaluation: $\mathbb{E}[R + \gamma V(S') s]$
Sarsa: $R + \gamma Q(S', A')$	Q-policy iteration: $\mathbb{E}[R + \gamma Q(S', A') s, a]$
Q-learning: $R + \gamma \max_{a'} Q(S', a')$	Q-value iteration: $\mathbb{E}[R + \gamma \max_{a'} Q(S', a') s, a]$

Next class: n -step Bootstrapping (Chapter 7 of Sutton & Barto)